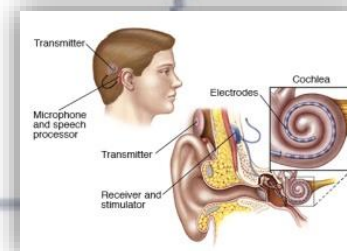
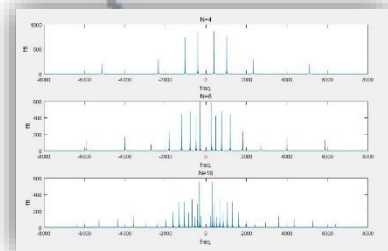
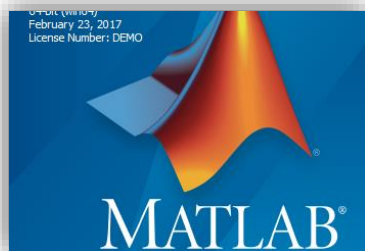


信号与系统实验

主讲人：吴光 博士

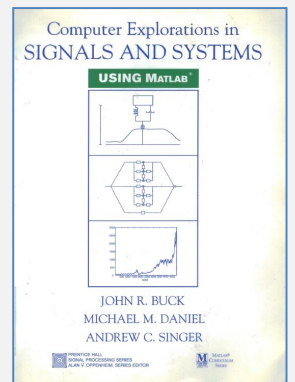
Email: wug@sustech.edu.cn



Labs in this course

5 Lab assignments+2 Projects

- Lab 1: MATLAB Programming (3 Weeks)
- **Lab 2: Linear Time-Invariant Systems (2 Weeks)**
- Lab 3: Fourier Series Representation of Periodic Signals (2 Weeks)
- Lab 4: The Continuous-Time Fourier Transform (2 Weeks)
- Lab 5: Coding Test & Low-Pass Filter (1 Week)
- Project 1: Motion detection via communication signals (3 Weeks)
- Project 2: Modulation Classification with Deep Learning (3 Weeks)



Signals and Systems (Lab)

Lab 2: Linear Time-Invariant Systems

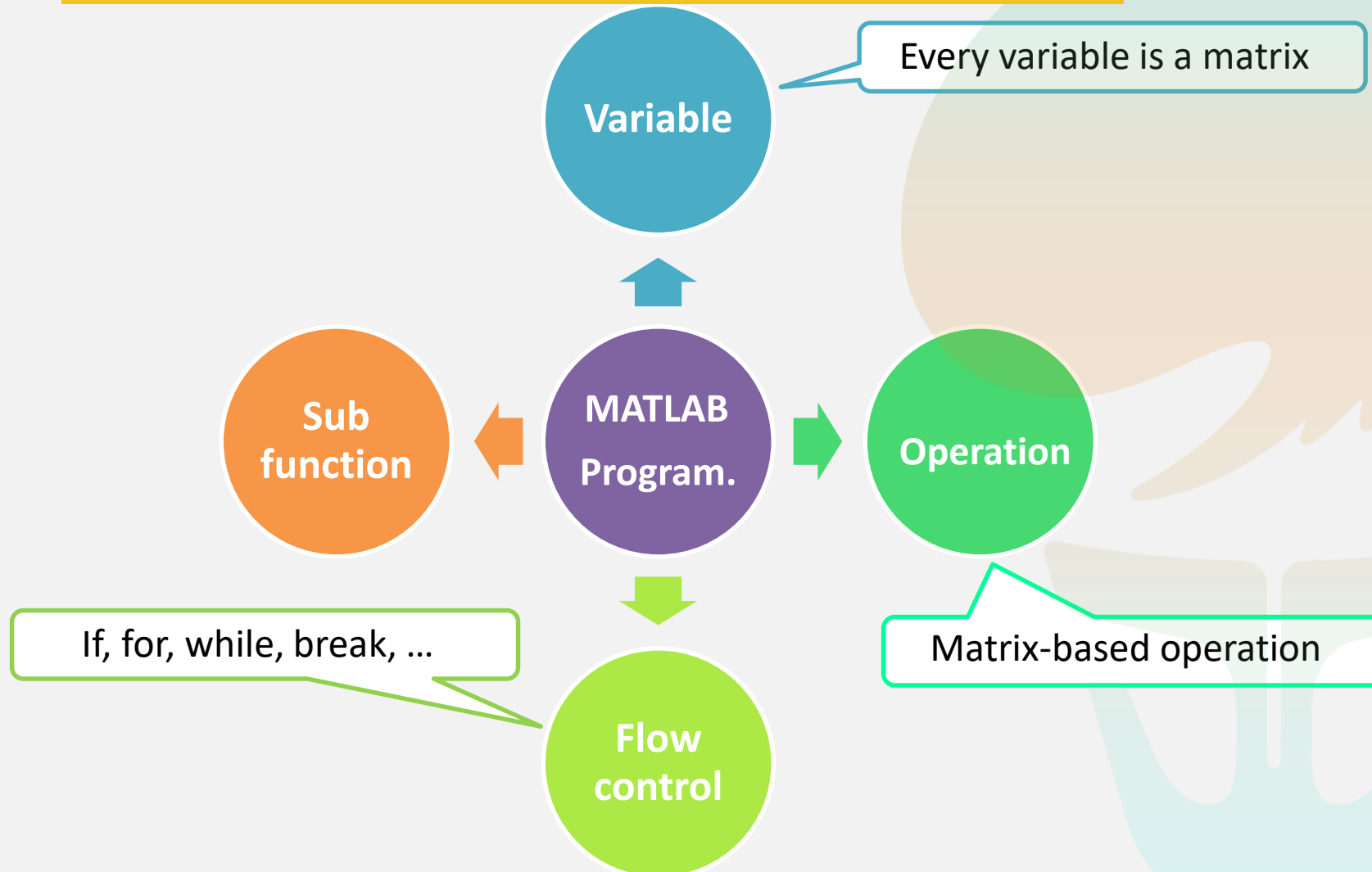
Part 1

Dr. **Wu Guang**

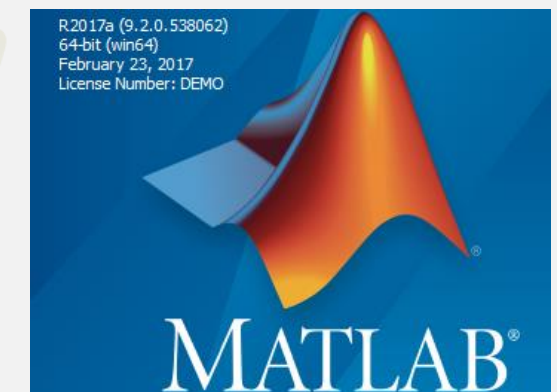
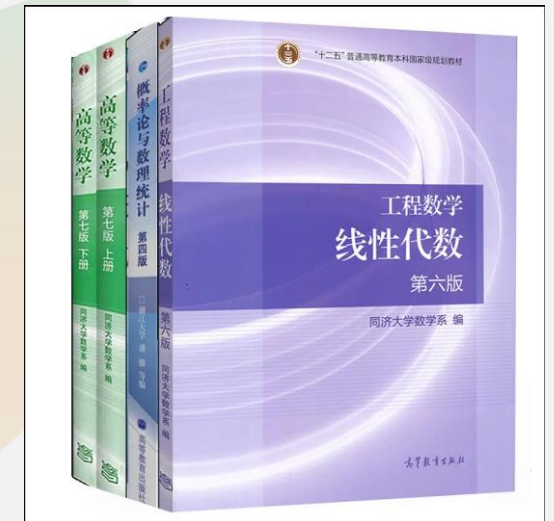
wug@sustech.edu.cn

Electrical & Electronic Engineering
Southern University of Science and Technology

1.0 Preliminary knowledge



Linear algebra



1.1 How to calculate the output of the system ?

- According to the properties of the system, e.g. Linearity, Time-Invariance.
- Step 1: to measure the system function:



- Step 2: to represent the $x(t)$ as a combination of the $\delta(t)/\delta(t - kT_s)$:



Requirement: H is a LTI system

1.2 The definition of **convolution**

- The system H is Linear Time Invariant (LTI):



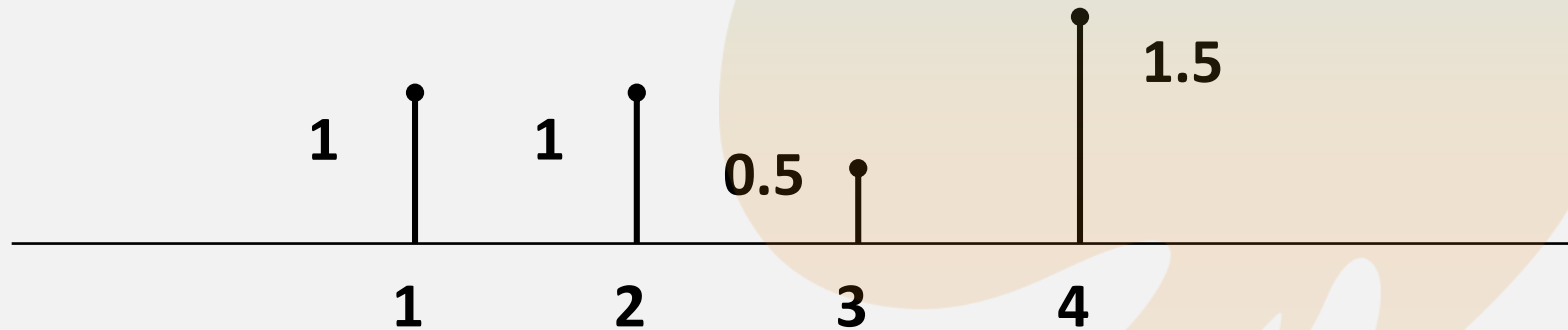
- The definition of convolution:

Linear Convolution

$$y(t) = x(t) \circledast h(t) = \int_{-\infty}^{\infty} x(\tau) h(t - \tau) d\tau$$

1.3 Exercise 1: Discrete-Time signal

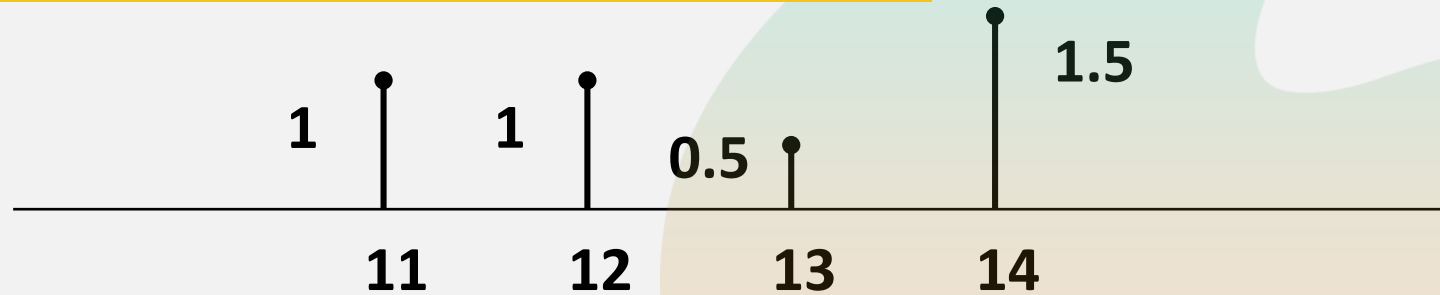
➤ Discrete-Time signal (DT signal):



➤ MATLAB code:

```
x=[1 1 0.5 1.5]  
stem(x)
```

1.4 Another DT signal



➤ MATLAB code:

```
x=[1 1 0.5 1.5]  
stem([11 12 13 14], x)
```

Signal value

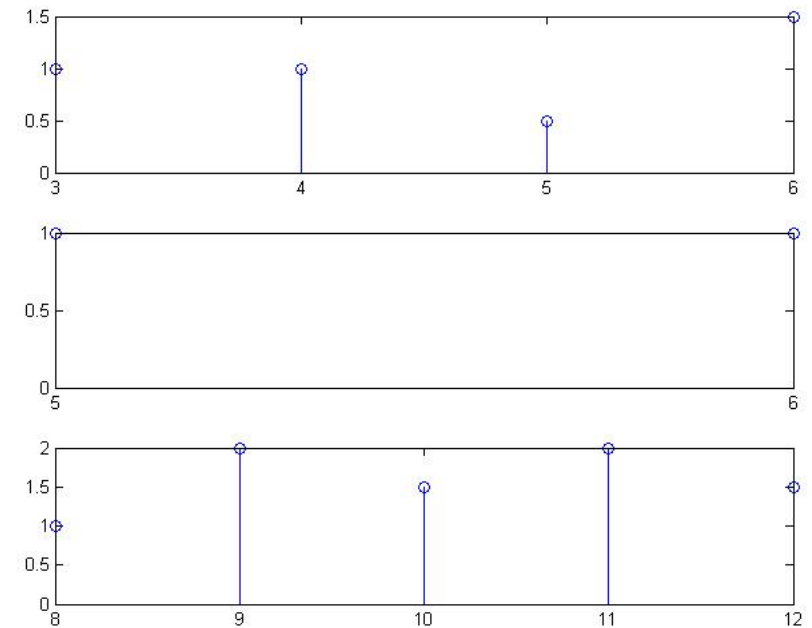
```
x=[1 1 0.5 1.5]  
nx=[11 12 13 14]  
stem(nx, x)
```

Non-zero region
of time index

1.5 Exercise 2: Calculate Convolution by conv()

- Step 1: Define u , nu and v , nv
- Step 2: Calculate the signal values of $w[n]$ by **$w=\text{conv}(u,v)$**
- Step 3: Calculate the non-zero interval of $w[n]$ by **$nw=nu(1)+nv(1):nu(\text{end})+nv(\text{end})$**

```
u=[1 1 0.5 1.5]
nu=3:6
v=[1 1]
nv=5:6
w=conv(u,v)
nw=nu(1)+nv(1):nu(end)+nv(end)
subplot(3,1,1), stem(nu,u)
subplot(3,1,2), stem(nv,v)
subplot(3,1,3), stem(nw,w)
```



1.6 DT Signal Convolution: Flip->Shift->Multiplication->Summation

- $w[n] = u[n] * v[n] = \sum_{k=-\infty}^{+\infty} u[k]v[n-k]$
- Non-zero interval of DT signal u : **$nu=a:a+x$**
 - Non-zero interval of DT signal v : **$nv=b:b+y$**
 - Non-zero interval of $w[n]$: **$nw=? : ??$**

$$n-b=a \Rightarrow n=a+b$$

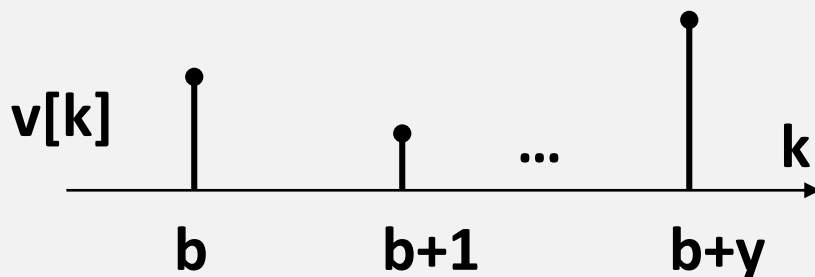
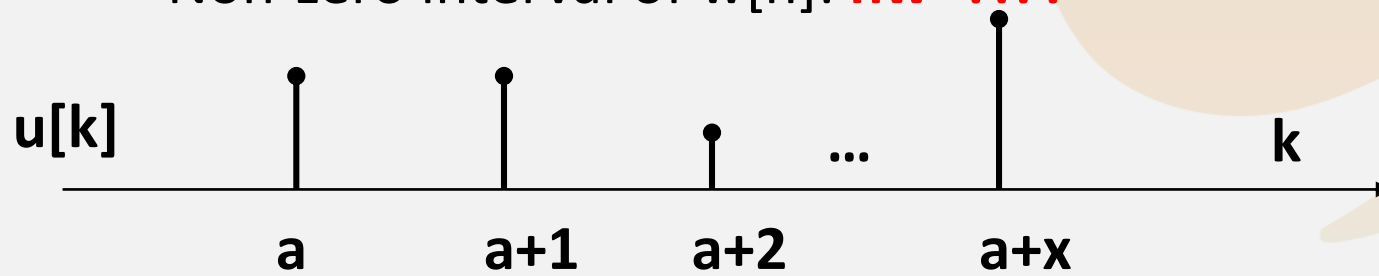
$$n-b-y=a+x \Rightarrow n=a+b+y+x$$

Hence,

$$nw=[a+b:a+b+x+y]$$

$$? = a+b = nu(1) + nv(1)$$

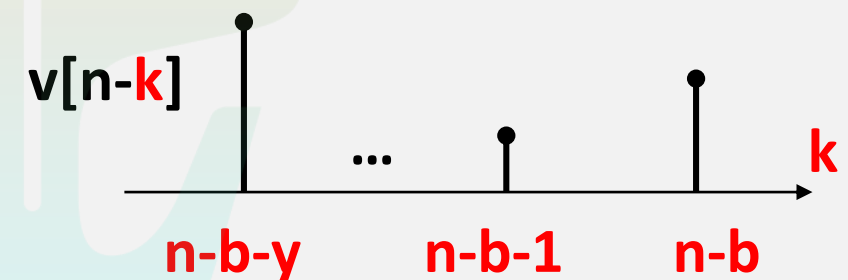
$$?? = a+b+x+y = nu(\text{end}) + nv(\text{end})$$



Flip w.r.t the origin



Shift to the right by n



2.1 LTI System by Difference Equation

- Generally,
- **Causal** DT LTI system can be specified by a linear constant-coefficient difference equation:

$$\sum_{k=0}^K a_k y[n-k] = \sum_{m=0}^M b_m x[n-m]$$

输出

- The left summation is output, the right summation is input.
- In MATLAB, a causal DT LTI system is uniquely specified by two vectors:
 $A = [a_0 \ a_1 \ a_2 \ \cdots \ a_K]$ and $B = [b_0 \ b_1 \ b_2 \ \cdots \ b_M]$

2.2 Impulse Response

- Particularly, $y[n]$ will be impulse response as the input is impulse function
 - $y[n]=0.5x[n]+x[n-1]+2x[n-2]$;
 $h[n]=?$ Finite Impulse Response (FIR)
 - $y[n]-0.8y[n-1]=2x[n]$;
 $h[n]=?$ Infinite Impulse Response (IIR)
- **Causal** DT LTI system is uniquely specified by two vectors: $A=[a_0 \ a_1 \ a_2 \ \dots \ a_K]$ and $B=[b_0 \ b_1 \ \dots \ b_M]$
 - $A=[1] \ B=[0.5 \ 1 \ 2]$
 - $A=[1 \ -0.8] \ B=[2]$

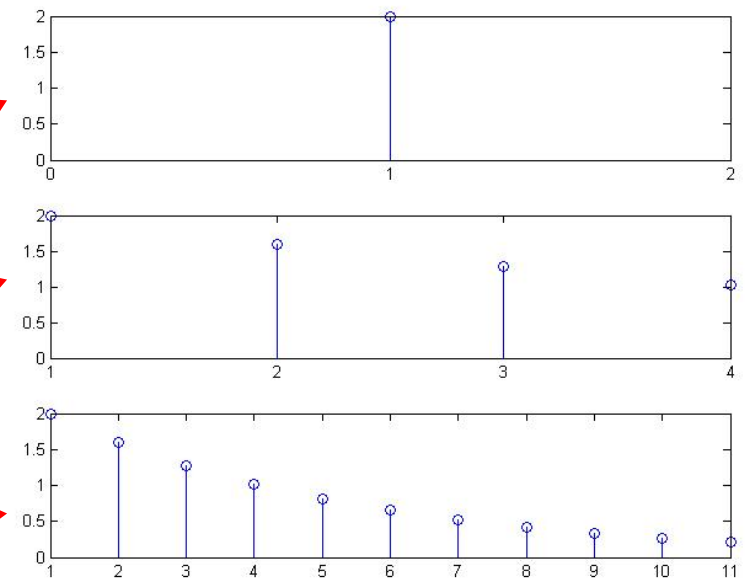
$$\sum_{k=0}^K a_k y[n-k] = \sum_{m=0}^M b_m x[n-m]$$

2.3 Exercise 3: Calculate Output Signal by filter()

- Syntax: $y = \text{filter}(B, A, x)$
- System: specify by the coefficient vector A and B
- x and y share the same range of time indices
- Output signal y may be truncated

$$y[n] - 0.8y[n-1] = 2x[n]$$

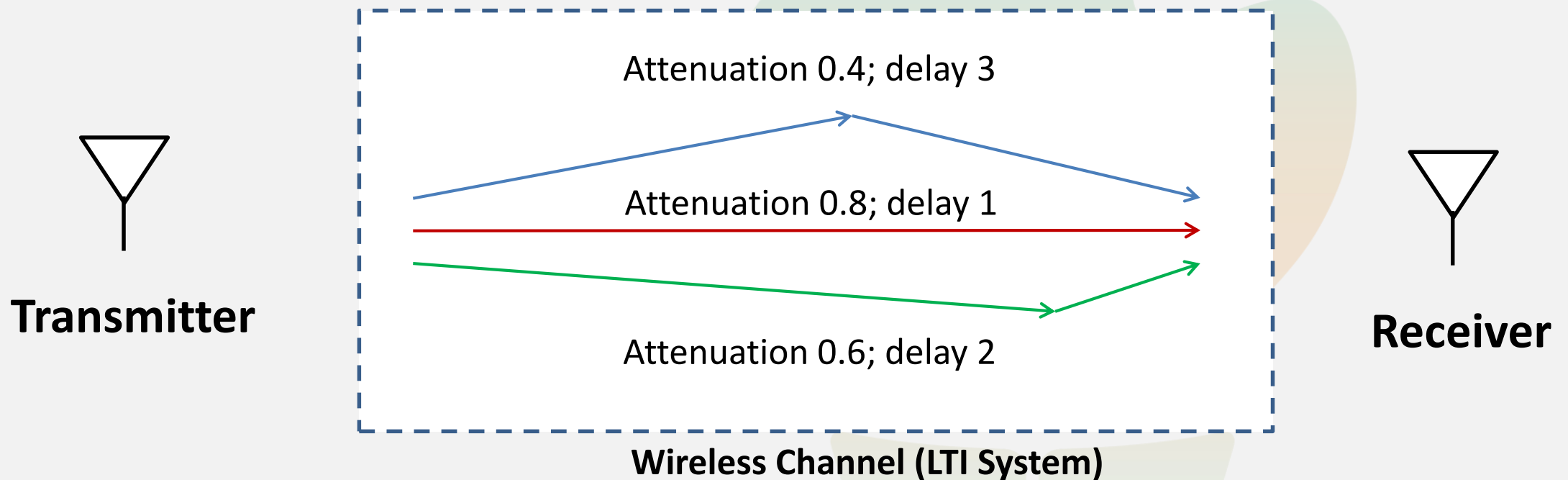
```
A = [1 -0.8]
B = 2
X1 = [1]
Y1 = filter(B, A, X1);
X2 = [1 0 0 0]
Y2 = filter(B, A, X2);
X3 = [1 0 0 0 0 0 0 0 0 0]
Y3 = filter(B, A, X3);
subplot(3,1,1), stem(Y1)
subplot(3,1,2), stem(Y2)
subplot(3,1,3), stem(Y3)
```



2.4 conv and filter

	conv	filter
Scenario	• Finite signal • Finite impulse response	• Finite signal • Infinite/finite impulse response
Remarks	• $nw = nu(1) + nv(1) : nu(end) + nv(end)$	• x and y share the same range of time indices • Truncated output

3.1 Application: Simplified Wireless Channel



Impulse response: $h_1[n] = 0.8\delta[n-1] + 0.6\delta[n-2] + 0.4\delta[n-3]$

Difference equation: $y[n] = 0.8x[n-1] + 0.6x[n-2] + 0.4x[n-3]$

3.2 How to recover the transmission signal ?

Generate Tx Signal:

```
x=[1 2 3 4 0 0 1 3 2 1 0 0 0];
```

Generate Rx Signal:

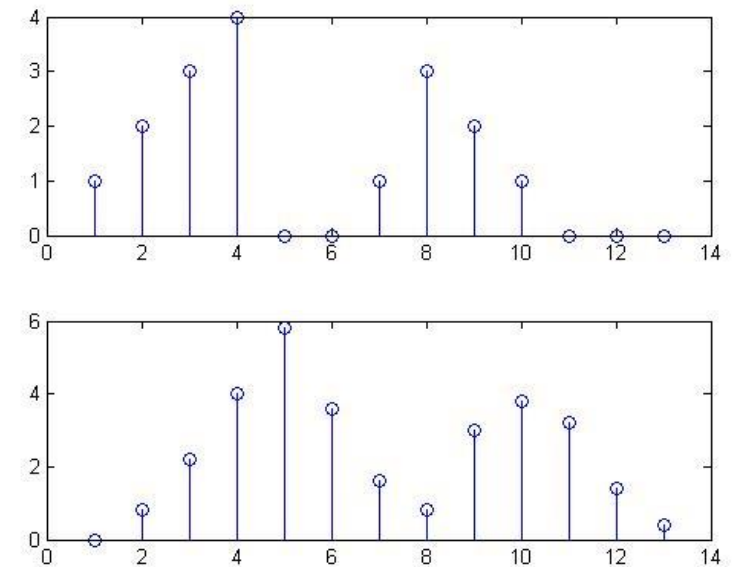
```
A1 = 1;
```

```
B1 = [0 0.8:-0.2:0.4];
```

```
y = filter(B1, A1, x);
```

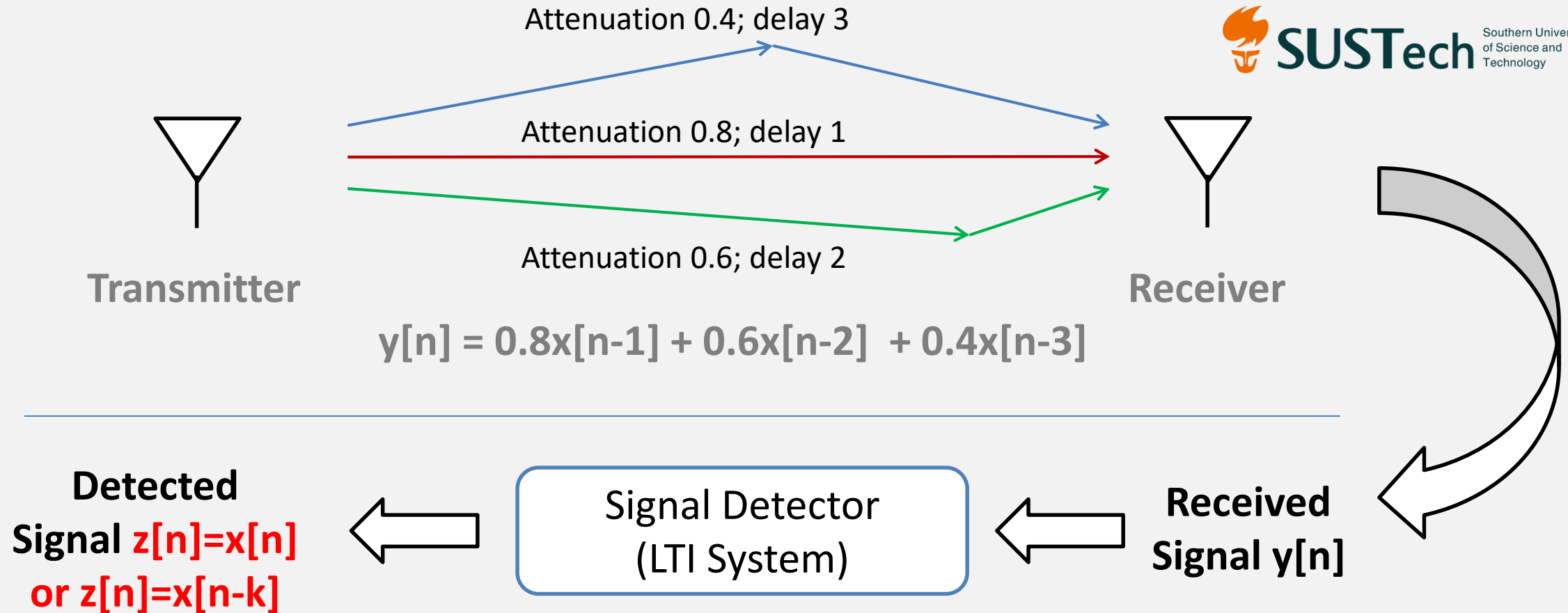
```
subplot(2,1,1), stem(x);
```

```
subplot(2,1,2), stem(y);
```



Signal detection:

How to recover the transmission signal from the received signal ?



Impulse response: $h_2[n] * h_1[n] = \delta[n] \text{ or } \delta[n-k]$

Difference Equation: $0.8z[n-1] + 0.6z[n-2] + 0.4z[n-3] = y[n]$

$$z'[n] = z[n-1] \Rightarrow 0.8z'[n] + 0.6z'[n-1] + 0.4z'[n-2] = y[n]$$

3.3 Compare the two signals

Generate Detected Signal:

```
A2 = 0.8:-0.2:0.4;
```

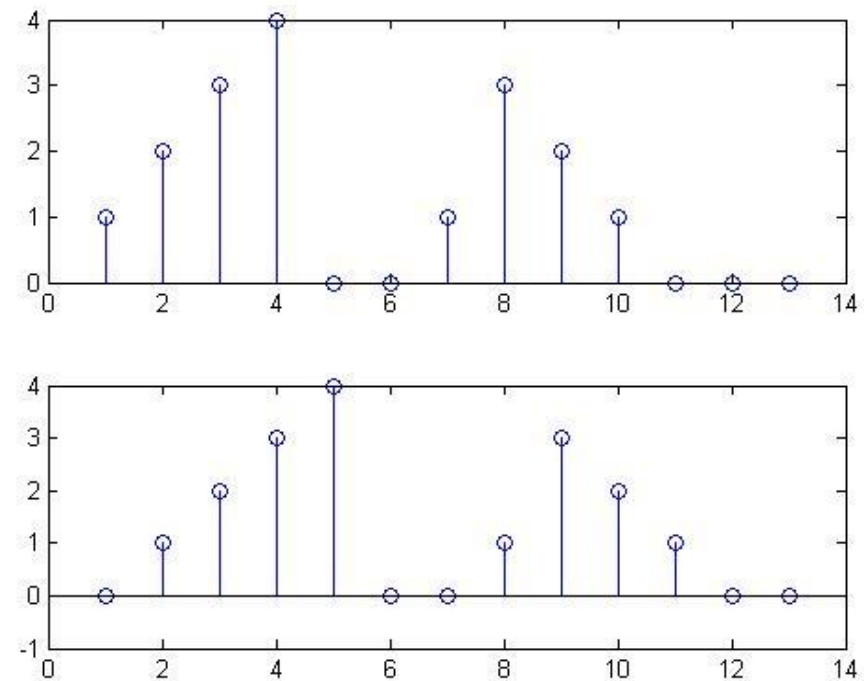
```
B2 = 1;
```

```
z = filter(B2, A2, y);
```

Compare the two signals:

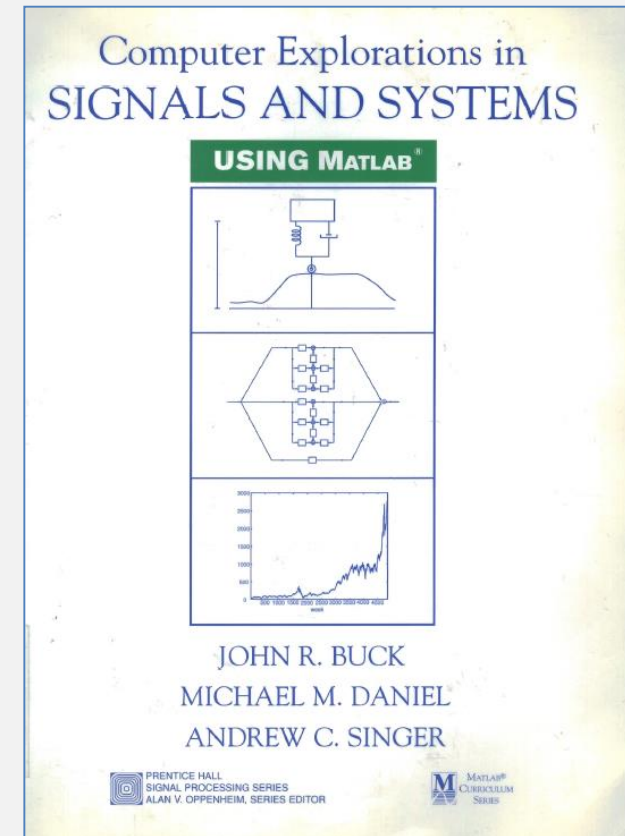
```
subplot(2,1,1), stem(x);
```

```
subplot(2,1,2), stem(z);
```



Lab Assignment

- Read tutorial 2.1 & 2.2 & 2.3 & 2.4 by yourself
- 2.10
 - The sound file for 2.10: **lineup.mat**
- Submit your report.



Question ?

